

Pour l'exo 1 de la série 1 On va considérer que les qst sont indépendantes .

Remarque : Chaque tableau a une taille maximal à ne pas dépasser , Attention la taille max $\neq$ Taille réel du tableau . La taille réel est celle saisie par l'utilisateur elle doit être sup a 0 et inferieur ou egal a la taille max .

La Taille max est soit donnée directement , sinon on prend une valeur au hasard pour notre exo on va prendre $taille_max=20$

Exp : Un tableau peut contenir jusqu'à 20 éléments , ceci est la taille du tableau , la taille réel est le nombre saisie par l'user il peut être 5 , 10 , 20, 15 ... $n > 0$ et $n \leq 20$.

Pour chaque algorithme il est impérative de demander la taille réel que l'utilisateur veut saisir

+ Remplir le tableau car on ne peut appliquer aucun traitement sur un tableau vide , attention des fois le remplissage du tableau n'est pas demander il est fait d'une fois implicite

Algorithm	Program C
Algorithm qst_1 Var n, i : integer; A : ARRAY[20] of integer; Begin DO begin PRINT("Give n > 0 : "); SCAN(n); end WHILE (n <= 0 OR n > 20); for i ← 0 to n-1 step 1 do begin PRINT("Give element A[" , i , "] : "); SCAN(A[i]); end End	<pre>#include <stdio.h> int main() { int A[20]; int n, i; do { printf("Give n > 0 : "); scanf("%d", &n); } while (n <= 0 n > 20); for (i = 0; i < n; i++) { printf("Give element A[%d] : ", i); scanf("%d", &A[i]); } return 0; }</pre>
Algorithm	Program C
Algorithm qst_2_3 Var n, i , c : integer; A : ARRAY[20] of integer; Begin do begin PRINT("Give n > 0 : "); SCAN(n); end while (n <= 0 OR n > 20); // Lire la constante print("Give the constant c : "); scan(c); // Remplir le tableau avec la constante	<pre>#include <stdio.h> int main() { int n, c, i; int A[20]; // Tableau de taille maximale 20 // Lire n la taille réel du tableau do { printf("Give n > 0 (max 20): "); scanf("%d", &n); } while (n <= 0 n > 20); // Lire la constante printf("Give the constant c: "); scanf("%d", &c); }</pre>

<pre> for i ← 0 to n-1 step 1 do begin A[i] ← c ; end // Afficher le tableau for i ← 0 to n-1 step 1 do begin A[i] ← c ; end End </pre>	<pre> // Remplir le tableau avec la constante for(i = 0; i < n; i++) { A[i] = c ; } // Afficher le tableau printf("The array A is: "); for(i = 0; i < n; i++) { printf("%d ", A[i]); } printf("\n"); return 0; } </pre>
Algorithm	Program C
Algorithm qst_4 Var n, i, x : integer; A : ARRAY[20] of integer; Begin do begin PRINT("Give n > 0 : "); SCAN(n); end while (n <= 0 OR n > 20); // Remplir le tableau for i ← 0 to n-1 step 1 do begin print("Give element A[" , i, "] : "); scan(A[i]); End // Lire la variable X print("Give the element x : "); scan(x); // boucle de recherche de valeur <pre> exist ← false ; for i ← 0 to n-1 step 1 do begin if(A[i]= x) then begin existe ← true ; end end </pre> if(existe = true) then begin print("The value existe"); end else begin print("the values does not exist"); end End	<pre> #include <stdio.h> #include<stdbool.h> int main() { int n,x,i; int A[20]; // Tableau de taille maximale 20 bool exist ; // Lire n la taille réel du tableau do { printf("Give n > 0 (max 20): "); scanf("%d", &n); } while (n <= 0 n > 20); // Remplir le tableau for (i = 0; i < n; i++) { printf("Give element A[%d] : ", i); scanf("%d", &A[i]); } // Lire la variable X printf("Give the element x : "); scanf("%d",&x); // boucle de recherche de valeur exist = false ; for(i=0 ; i<n ; i++){ if (A[i] == x) { exist = true ; } } if (exist==true) { printf("The value exist"); } else { printf("The value does not exist"); } return 0; } </pre>

La Boucle la plus optimisé pour la recherche

<pre> while(i<n and exist = false) do begin if(A[i]= x) then begin existe ← true ; end i←i+1 ; end end </pre>	<pre> while (i<n && exist == false) { if(A[i]== x) { exist = true ; } i++ ; } </pre>
--	---

Algorithm	Program C
Algorithm qst_5 Var n, i : integer; A : ARRAY[20] of integer; Begin DO begin PRINT("Give n> 0 : "); SCAN(N); end WHILE (n <= 0 OR n > 20); for i ← 0 to n-1 step 1 do begin PRINT("Give element A[" , i, "] : "); SCAN(A[i]); end // Lire la variable X print("Give the element x : "); scan(x); // Initialiser le compteur cpt ← 0 // Compter les occurrences for i ← 0 to n-1 step 1 do begin if(A[i]= x) then begin cpt ← cpt+1 ; end end end // Afficher le résultat print(x , "occurs" , cpt , " times in the array"); End	<pre> #include <stdio.h> int main() { int A[20]; int n,x, i, cpt ; do { printf("Give N > 0 : "); scanf("%d", &n); } while (n <= 0 n > 20); for (i = 0; i < n; i++) { printf("Give element A[%d] : ", i); scanf("%d", &A[i]); } // Lire l'élément à compter printf("Give the element X to count: "); scanf("%d", &x); // Initialiser le compteur cpt = 0; // Compter les occurrences for(i = 0; i < n; i++) { if(A[i] == x) cpt++; } // Afficher le résultat printf("%d occurs %d times in the array.\n", x, cpt); return 0; } </pre>

Algorithm	Program C
Algorithm qst_6 Var n, i , pos ,neg , zero: integer; A : ARRAY[20] of integer; Begin DO begin PRINT("Give n> 0 : "); SCAN(N); end WHILE (n <= 0 OR n > 20); for i ← 0 to n-1 step 1 do begin PRINT("Give element A[" , i , "] : "); SCAN(A[i]); end // Initialiser les compteurs pos←0 ; neg←0 ; zero←0 // Compter les occurrences for i ← 0 to n-1 step 1 do begin if(A[i] == 0)then begin zero++; end else if(A[i] > 0) then begin pos++; end else begin neg++; end end end print("The number of zeros is " , zero); print("The number of positive integers is " , pos); print("The number of negative integers is " , neg); End	<pre>#include <stdio.h> int main() { int n , i , zero ,pos,neg; int A[20]; // Lire la taille du tableau do { printf("Give N > 0 (max 20): "); scanf("%d", &n); } while (n <= 0 n > 20); // Lire les éléments du tableau for(i = 0; i < n; i++) { printf("Give element A[%d]: ", i); scanf("%d", &A[i]); } // Compter les positives, négatives et zéros for(i = 0; i < n; i++) { if(A[i] == 0) zero++; else if(A[i] > 0) pos++; else neg++; } // Afficher les résultats printf("The number of zeros is %d\n", z); printf("The number of positive integers is %d\n", p); printf("The number of negative integers is %d\n", n); return 0;} </pre>

ALGORITHM qst7**VAR**

N, i, min : INTEGER;
A : ARRAY[20] OF INTEGER;

BEGIN**DO**

PRINT("Give N > 0 : ");
SCAN(N);

WHILE (N <= 0 OR N > 20);

// Lire les éléments du tableau

FOR i ← 0 TO N-1 DO

BEGIN

PRINT("Give element A[" , i, "]" : ");
SCAN(A[i]);

END

// Initialiser le minimum avec le premier élément

min ← A[0];

// Parcourir le tableau pour trouver le minimum

FOR i ← 1 TO N-1 DO

BEGIN

IF A[i] < min THEN
min ← A[i]

END

// Afficher le résultat

PRINT("The minimum of the array is " , min)

END

```
#include <stdio.h>
```

```
int main() {
```

```
int N, i, min;
```

```
int A[20];
```

// Lire la taille du tableau

```
do {
```

```
printf("Give N > 0 (max 20): ");
```

```
scanf("%d", &N);
```

```
} while (N <= 0 || N > 20);
```

// Lire les éléments du tableau

```
for(i = 0; i < N; i++) {
```

```
printf("Give element A[%d]: " , i);
```

```
scanf("%d", &A[i]);
```

```
}
```

// Initialiser le minimum avec le premier élément

```
min = A[0];
```

// Parcourir le tableau pour trouver le minimum

```
for(i = 1; i < N; i++) {
```

```
if(A[i] < min)
```

```
min = A[i];
```

```
}
```

// Afficher le résultat

```
printf("The minimum of the array is %d\n", min);
```

```
return 0;
```

```
}
```

ALGORITHM qst8**VAR**

N, i, M : INTEGER;
A : ARRAY[20] OF INTEGER;

BEGIN**DO**

PRINT("Give N > 0 : ");
SCAN(N);

WHILE (N <= 0 OR N > 20);

// Lire les éléments du tableau

FOR i ← 0 TO N-1 DO

BEGIN

PRINT("Give element A[" , i, "]" : ");
SCAN(A[i]);

END

// Initialiser le maximum avec le premier

```
#include <stdio.h>
```

```
int main() {
```

```
int N, i, M;
```

```
int A[20];
```

// Lire la taille du tableau

```
do {
```

```
printf("Give N > 0 (max 20): ");
```

```
scanf("%d", &N);
```

```
} while (N <= 0 || N > 20);
```

// Lire les éléments du tableau

```
for(i = 0; i < N; i++) {
```

```
printf("Give element A[%d]: " , i);
```

```
scanf("%d", &A[i]);
```

```
}
```

élément $M \leftarrow A[0];$ // Parcourir le tableau pour trouver le maximum FOR $i \leftarrow 1$ TO $N-1$ DO BEGIN IF $A[i] > M$ THEN $M \leftarrow A[i]$ END // Afficher le résultat PRINT("The maximum of the array is ", M) END	// Initialiser le maximum avec le premier élément $M = A[0];$ // Parcourir le tableau pour trouver le maximum for($i = 1; i < N; i++$) { if($A[i] > M$) $M = A[i];$ } // Afficher le résultat printf("The maximum of the array is %d\n", M); return 0; }
---	--

Par rapport a la déviation (qst 9) :

Deviation = max – min

➔ Donc calculez le max et le min du tableau , ensuite calculez leur difference